

Integración Numérica

Tema 6. Integración numérica

Cálculo Numérico
Diego Rojo



Motivación

¿Por qué integrar numéricamente?

- Las integrales aparecen por todas partes en ciencia e ingeniería:
 - **Mecánica clásica:** trabajo $W = \int_{x_0}^{x_1} F(x) dx$, energía cinética, momento de inercia.
 - **Probabilidad:** $P(a \leq X \leq b) = \int_a^b f_X(x) dx$.
 - **Electromagnetismo / mecánica cuántica:** integrales de campos y de funciones de onda.

- Muchas integrales **no admiten primitiva en forma cerrada**:

$$\int_a^b \sqrt{1 + \cos^2(x)} dx, \quad \int_0^x e^{-t^2} dt, \quad \int_a^b \frac{1}{\ln x} dx.$$

- En otros casos, sólo tenemos **f tabulada** $(x_i, f(x_i))$ a partir de mediciones o de una simulación costosa.

Cuadratura

El problema general

Aproximar la integral

$$I[f] = \int_a^b f(x) dx$$

por una **suma finita** de la forma

$$I_N[f] = \sum_{i=0}^N w_i f(z_i).$$

- Los w_i son los **pesos** (*weights*).
- Los $z_i \in [a, b]$ son los **nodos** (*quadrature points* o *nodes*).
- La fórmula se llama **regla de cuadratura** (*quadrature rule*).
- Idealmente, $\lim_{N \rightarrow \infty} I_N[f] = I[f]$.

¿De dónde viene la idea?

De la propia **definición** de la integral de Riemann: dada una partición

$a = x_0 < x_1 < \cdots < x_{N+1} = b$ y puntos $z_i \in [x_i, x_{i+1}]$,

$$\int_a^b f(x) dx = \lim_{N \rightarrow \infty} \sum_{i=0}^N (x_{i+1} - x_i) f(z_i).$$

- Tomamos un N **finito** y nos quedamos con una suma.
- Distintas reglas de cuadratura $\Leftrightarrow$ distintas formas de elegir los z_i y los w_i .

Regla del Punto Medio

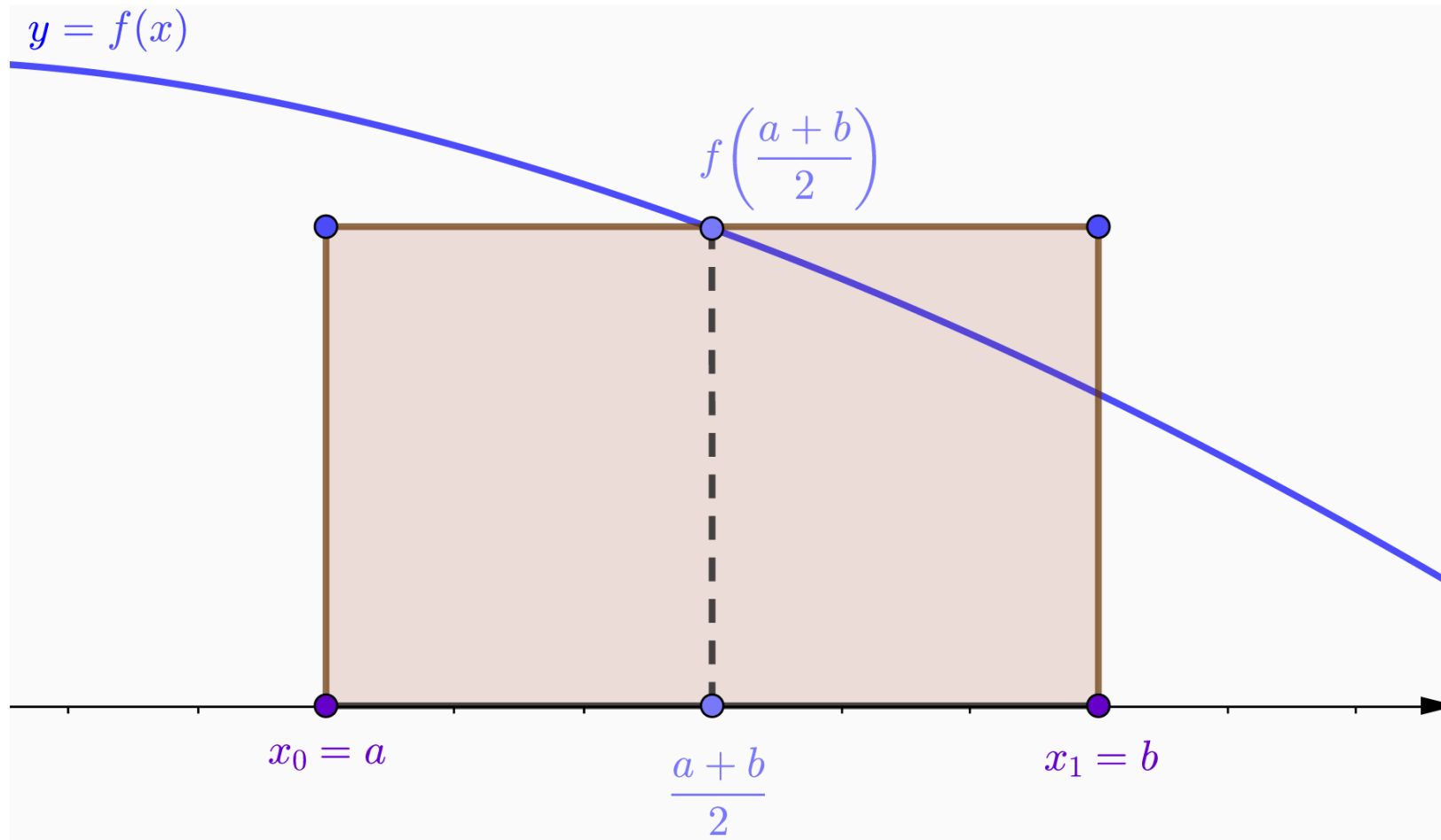
Regla del punto medio (simple)

- Caso más sencillo: **un único intervalo** $[a, b]$ y **un único nodo**, el punto medio.
- Aproximamos el área bajo f por el área del **rectángulo** de base $b - a$ y altura $f((a + b)/2)$:

$$I[f] \approx I_0[f] = (b - a) f\left(\frac{a + b}{2}\right)$$

- Peso único: $w_0 = b - a$. Nodo único: $z_0 = (a + b)/2$.
- Si $b - a$ es grande el error puede ser inaceptable; se emplea la **versión compuesta**.

Regla del punto medio (simple)



Regla del punto medio (compuesta)

- Subdividimos $[a, b]$ en N subintervalos (paneles) de longitud $h = (b - a)/N$:

$$a = x_0 < x_1 < \cdots < x_N = b, \quad x_i = a + i h.$$

- Aplicamos la regla simple en cada subintervalo $[x_i, x_{i+1}]$:

$$\int_{x_i}^{x_{i+1}} f(x) dx \approx h f\left(\frac{x_i + x_{i+1}}{2}\right).$$

- Sumando:

$$I_N[f] = h \sum_{i=0}^{N-1} f\left(\frac{x_i + x_{i+1}}{2}\right)$$

Idea operativa: regla compuesta

Regla compuesta: cuando dividimos el intervalo $[a, b]$ en subintervalos $[x_i, x_{i+1}]$ y aplicamos una regla de cuadratura **en cada subintervalo**, la regla resultante se llama **compuesta** (*composite*).

- La estrategia es **idéntica** para todas las reglas que veremos: trapecio compuesto, Simpson compuesto, ...
- La regla simple se reserva para subintervalos pequeños; la compuesta es la que se usa en la práctica.

Reglas de Newton-Cotes

Idea general

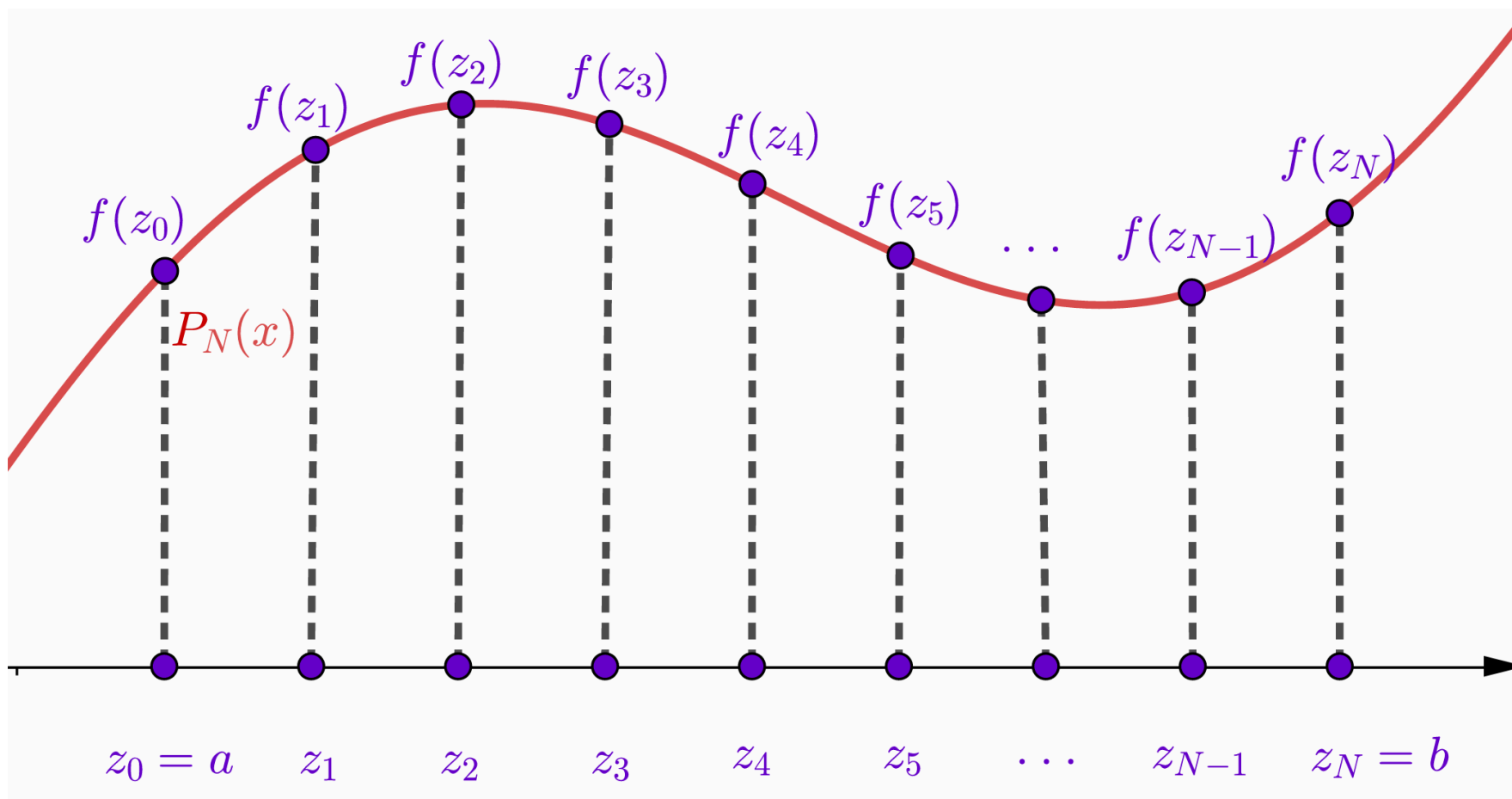
- Tomamos $N + 1$ nodos **equiespaciados** $a = x_0 < x_1 < \cdots < x_N = b$.
- Aproximamos f por su **polinomio interpolador de Lagrange**:

$$P_N(x) = \sum_{i=0}^N f(z_i) \ell_i(x), \quad \ell_i(x) = \prod_{\substack{j=0 \\ j \neq i}}^N \frac{x - x_j}{x_i - x_j}.$$

- Aproximamos la integral por la **integral del polinomio** (que sí sabemos calcular exactamente):

$$I_N[f] = \int_a^b P_N(x) dx = \sum_{i=0}^N \underbrace{\left[\int_a^b \ell_i(x) dx \right]}_{w_i} f(z_i).$$

Idea general



La estructura es siempre la misma

$$I_N[f] = \sum_{i=0}^N w_i f(z_i), \quad w_i = \int_a^b \ell_i(x) dx.$$

- Cambiando N obtenemos distintas reglas:
 - $N = 1 \Rightarrow$ **trapecio** (polinomio lineal).
 - $N = 2 \Rightarrow$ **Simpson** (polinomio cuadrático).
 - $N = 3, 4, \dots \Rightarrow$ Simpson 3/8, Boole, ...
- Los pesos w_i se calculan **una vez**, integrando los ℓ_i . No dependen de f .

Regla del Trapecio

Trapecio simple

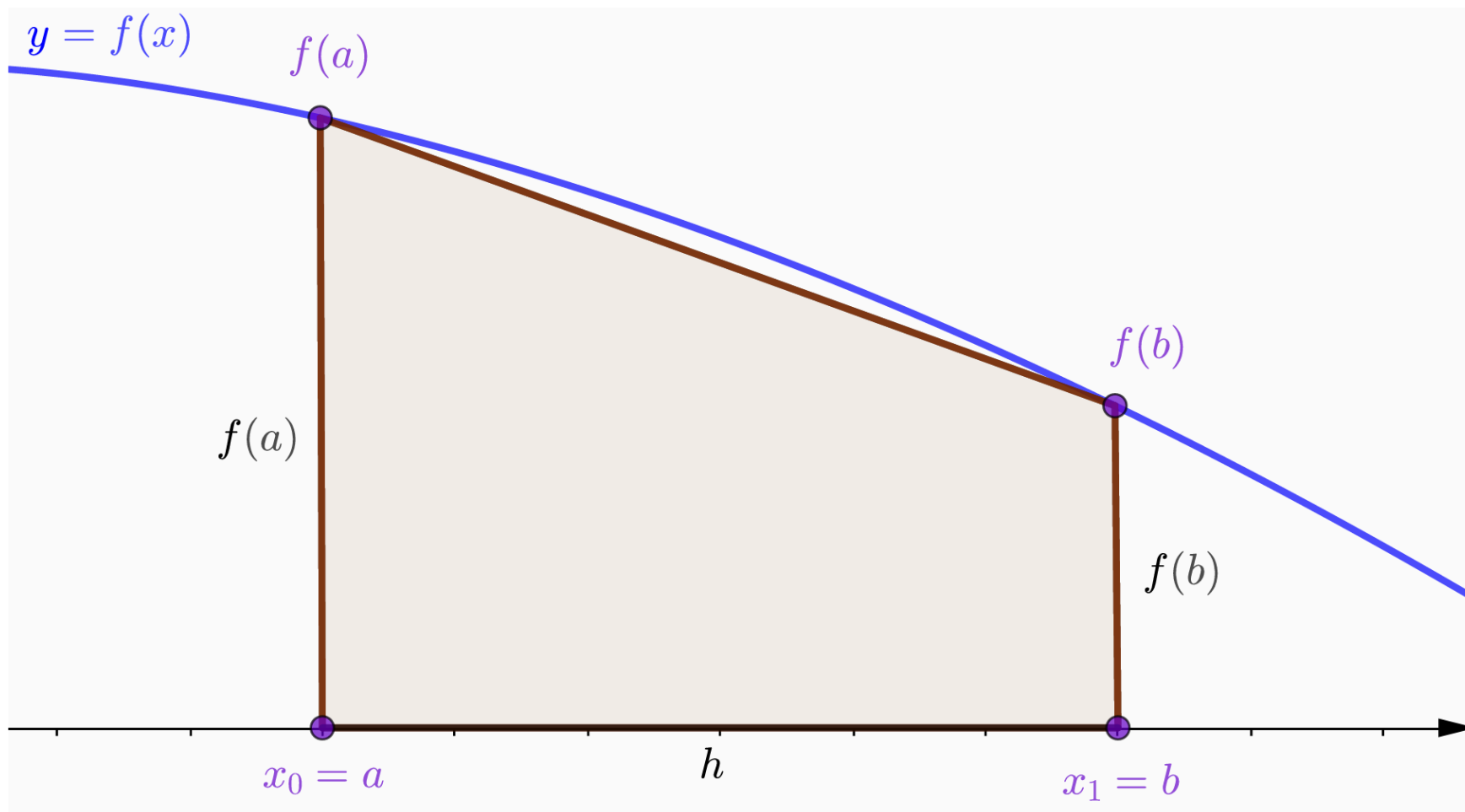
Newton-Cotes con $N = 1$: dos nodos $x_0 = a$, $x_1 = b$ y polinomio lineal P_1 .

Calculando $w_0 = \int_a^b \ell_0(x) dx$ y $w_1 = \int_a^b \ell_1(x) dx$ se obtiene $w_0 = w_1 = h/2$ con $h = b - a$.

$$I_1[f] = \frac{h}{2} [f(a) + f(b)]$$

- **Geometría:** área del trapecio bajo el segmento que une $(a, f(a))$ y $(b, f(b))$.
- Recordatorio: Área del trapecio = $\frac{(\text{base mayor} + \text{base menor}) \cdot \text{altura}}{2}$.

Trapecio simple



Error de la regla del trapecio (simple)

Del error de interpolación de Lagrange con $N = 1$:

$$f(x) - P_1(x) = \frac{f''(\xi)}{2!} (x - a)(x - b), \quad \xi \in (a, b).$$

Integrando entre a y b :

$$E_1[f] = \int_a^b [f(x) - P_1(x)] dx = -\frac{(b - a)^3}{12} f''(\xi).$$

- **Cota:** $|E_1[f]| \leq \frac{(b - a)^3}{12} \max_{[a,b]} |f''|.$
- El error escala como $(b - a)^3$: para intervalos amplios se emplea la versión **compuesta**.
- La regla del trapecio es **exacta** para polinomios de grado ≤ 1 .

Trapecio compuesto

Subdividimos $[a, b]$ en N paneles de longitud $h = (b - a)/N$ y aplicamos la regla simple en cada subintervalo $[x_i, x_{i+1}]$:

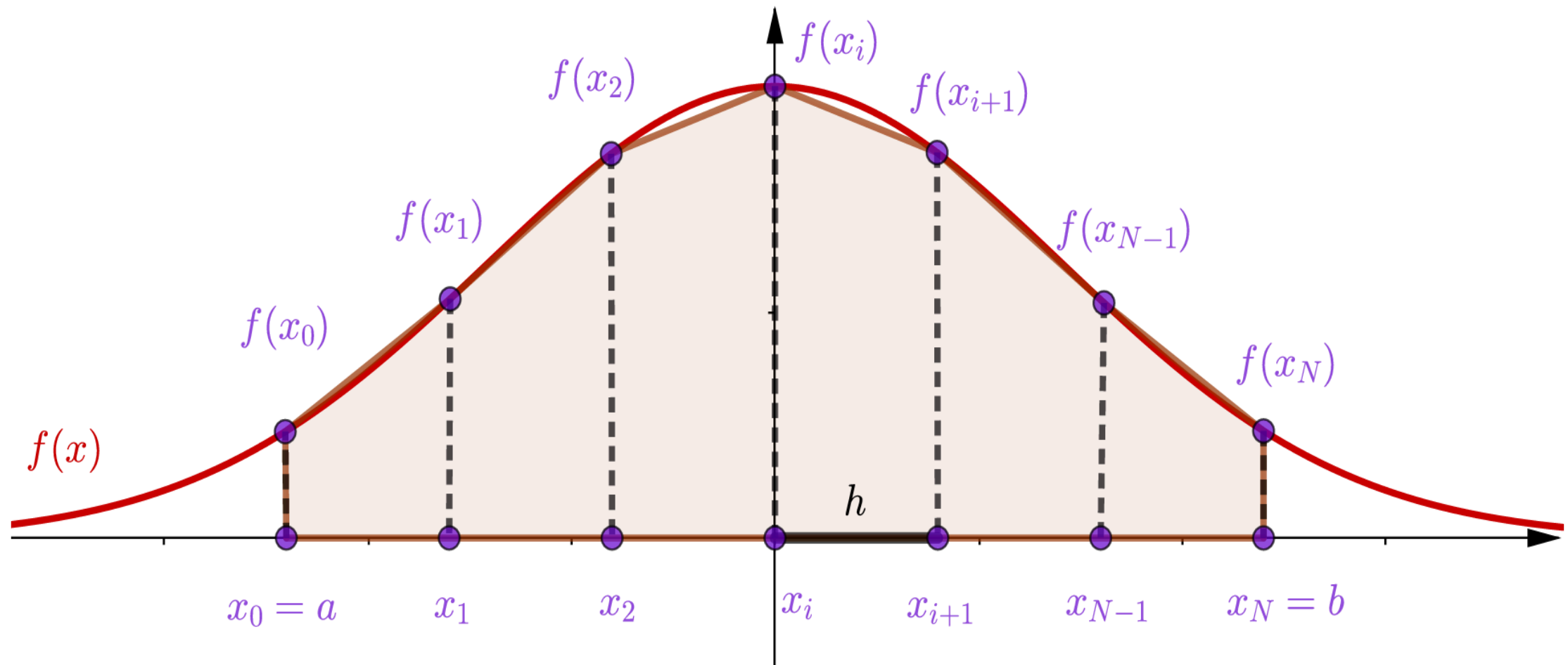
$$\int_{x_i}^{x_{i+1}} f(x) dx \approx \frac{h}{2} [f(x_i) + f(x_{i+1})].$$

Sumando los N paneles, los nodos **interiores** aparecen **dos veces**:

$$I_N[f] = \frac{h}{2} [f(x_0) + 2f(x_1) + 2f(x_2) + \cdots + 2f(x_{N-1}) + f(x_N)]$$

- Los extremos llevan peso $h/2$; los interiores, peso h .

Trapecio compuesto



Error del trapecio compuesto

Sumando los errores de los N paneles ($E_i = -\frac{h^3}{12} f''(\xi_i)$) y aplicando el TVM a f'' :

$$E_N[f] = -\frac{(b-a)h^2}{12} f''(\xi), \quad \xi \in (a, b).$$

- **Convergencia:** $E_N = O(h^2)$. Al dividir h entre 10 el error se divide entre 100.

Trapecio en `scipy.integrate`

`scipy.integrate.trapezoid(y, x=None, dx=1.0, axis=-1)` implementa la regla del trapecio compuesta sobre datos.

```
from scipy.integrate import trapezoid
import numpy as np

x = np.linspace(0.0, np.pi/2, 11)          # 11 nodos ⇒ N = 10 paneles
y = np.sin(x)
print(trapezoid(y, x))                     # ≈ 0.99794
```

- Recibe **valores** $y_i = f(x_i)$ y **nodos** x_i (o el paso `dx` si la malla es uniforme).
- Es la opción natural cuando sólo se dispone de **datos tabulados**.

Regla de Simpson

Simpson simple

Newton-Cotes con $N = 2$: tres nodos $x_0 = a$, $x_1 = (a + b)/2$, $x_2 = b$ y polinomio cuadrático P_2 .

Calculando $w_i = \int_a^b \ell_i(x) dx$ con el cambio $\xi = x - x_1$ se obtiene $w_0 = w_2 = h/3$, $w_1 = 4h/3$ con $h = (b - a)/2$.

$$I_2[f] = \frac{h}{3} \left[f(a) + 4f\left(\frac{a+b}{2}\right) + f(b) \right]$$

- Pesos 1, 4, 1 con factor común $h/3$.
- También llamada **regla 1/3 de Simpson**.

Simpson compuesto

Para aplicar Simpson en N subintervalos necesitamos un nodo **central** en cada par. Con $N = 2m$ par, dividimos $[a, b]$ en m pares de paneles $[x_{2k}, x_{2k+2}]$ y aplicamos Simpson simple en cada uno:

$$I_N[f] = \frac{h}{3} \left[f(x_0) + 4 \sum_{k \text{ impar}} f(x_k) + 2 \sum_{k \text{ par } 0 < k < N} f(x_k) + f(x_N) \right]$$

Esquema de pesos $\frac{h}{3} \cdot [1, 4, 2, 4, 2, \dots, 2, 4, 1]$.

- **Restricción importante:** N debe ser **par**.
- Coste: igual que el trapecio (mismo número de evaluaciones para igual malla).

Error de Simpson

- **Simple** (N=2):

$$E_2[f] = -\frac{(b-a)^5}{2^4 \cdot 180} f^{(4)}(\xi).$$

- **Compuesto** (N par):

$$E_N[f] = -\frac{(b-a) h^4}{180} f^{(4)}(\xi).$$

- **Convergencia:** $E_N = O(h^4)$. Al dividir h entre 10, el error se divide entre 10 000.
- Pasar del trapecio a Simpson reduce el error en **dos órdenes** aumentando sólo en uno el grado del polinomio interpolador. La razón se justifica con el grado de exactitud.

Simpson en `scipy.integrate`

`scipy.integrate.simpson(y, x=None, dx=1.0, axis=-1)` implementa la regla de Simpson compuesta.

```
from scipy.integrate import simpson
import numpy as np

x = np.linspace(0.0, np.pi/2, 11)          # 11 nodos  $\Rightarrow N = 10$  paneles (par)
y = np.sin(x)
print(simpson(y, x=x))                     #  $\approx 1.00000339$ 
```

- Misma firma que `trapezoid`: valores y_i y nodos x_i .
- Si N es impar, admite el argumento `even='first'/'last'/'avg'` para tratar el panel sobrante.

Grado de Exactitud

Definición

El **grado de exactitud** de una regla de cuadratura I_N es el mayor entero n tal que la regla integra **exactamente** todos los polinomios de grado $\leq n$:

$$E[x^k] = I[x^k] - I_N[x^k] = 0, \quad k = 0, 1, \dots, n,$$

y $E[x^{n+1}] \neq 0$.

- Como la integración es **lineal**, basta comprobar la propiedad sobre los monomios $1, x, x^2, \dots, x^n$.
- Una regla de Newton-Cotes con $N + 1$ nodos tiene grado de exactitud **al menos** N (integra exacto el propio polinomio interpolador, que es de grado N).

Trapezio y Simpson

- **Trapezio:** $N = 1$. Grado de exactitud **1**: integra exactamente constantes y rectas.
- **Simpson:** $N = 2$. Por construcción, grado **al menos 2**.
 - Adicionalmente integra exactamente x^3 por **simetría** alrededor del punto medio. Grado de exactitud **3**.
- La simetría aporta un orden adicional al esperado por el grado del polinomio interpolador, lo que justifica los dos órdenes extra de convergencia frente al trapezio.

¿Y si elegimos también los nodos?

- Hasta ahora: nodos **equiespaciados** (Newton-Cotes). Sólo controlamos los pesos.
- **Idea:** si dejamos también los z_i libres, tenemos $2(N + 1)$ parámetros y podemos imponer exactitud sobre $x^0, x^1, \dots, x^{2N+1}$.
 - Grado de exactitud **$2N + 1$** : casi el doble que en Newton-Cotes con el mismo número de evaluaciones.
- Esa familia se llama **cuadratura de Gauss**:
 - Implementada en `scipy.integrate.fixed_quad` y `quadrature`.
 - Los nodos óptimos son raíces de polinomios ortogonales (Legendre).

Resumen de reglas

Regla	Nodos	Pesos	Error compuesto	Grado de exactitud
Punto medio	1	h	$O(h^2)$	1
Trapecio	2 (extremos)	$h/2, h/2$	$O(h^2)$	1
Simpson	3 (extremos + medio)	$h/3, 4h/3, h/3$	$O(h^4)$	3
Gauss-Legendre	$N + 1$ (óptimos)	tabulados	—	$2N + 1$